Sustained Writes on Consumer Solid-State Drives

Erase geometry, discard semantics, and a clearing protocol for continuous microscopy acquisition

TRAPPY-SCOPES TECHNICAL NOTE · PREPARED FOR Y. BHASIN · 9 OCTOBER 2026 · SUBJECT: longterm/mjpeg_continuous_autosplit

ABSTRACT

We characterise the write workload of continuous MJPEG acquisition on a single microscope and relate it to the physical constraints of consumer NAND flash. Measured from acquisition timestamp files rather than estimated, the rig writes **629 kB per frame at 25.00 fps**, a sustained **15.74 MB/s**, giving 9.45 GB per ten-minute split and **765 GB** over an 81-split run. Three findings follow. First, this workload is close to the best case for flash: large sequential appends with whole-file deletion and no in-place update, so the random-write amplification models in the literature do not transfer to it and quoting them here would overstate the problem. Second, endurance is therefore not the binding constraint, the program/erase budget of a 1 TB TLC drive supporting on the order of a thousand full runs. Third, the risks that do bind are operational: the free-block pool is not restored by `rm`, and the dynamic SLC cache contracts sharply above about 80% fill, which an 81-split run reaches in its final hours. We give the mechanism by which deletion fails to reach the drive, the three standard discard commands, and a verification protocol for USB-attached drives on Raspberry Pi, where discard is frequently inoperative by default and silently so.

1 The measured workload

Every claim below is anchored to a measurement rather than to a datasheet figure. One sixty-second acquisition clip and its companion timestamp file give Table 1.

Quantity	Value	Derivation
Clip size	`944 452 088 B`	one .mjpeg split
Frames recorded	`1 501`	records in .tpts
Final timestamp	`59 992.115 ms`	last .tpts record
Mean interval	`39.995 ms`	/ 1 500 intervals
Frame rate	`25.003 fps`	nominal 25.000
Mean frame	`629.2 kB`	size / 1 501
Write rate	`15.74 MB/s`	size / 59.992 s
Per 600 s split	`9.45 GB`	15.74 MB/s x 600
Per 81-split run	`765 GB`	over 13.5 hours

TABLE 1 Workload measured on microscope M4 at 1520 x 1520, BGR888, quality 100.

Two features matter more than the headline number. The camera pins `FrameDurationLimits` at 40 000 us, and the measured mean interval of 39.995 ms agrees to one part in 10^4. The timestamp file holds 1 501 records where 1 501 are expected, so **no frames were dropped**: the encoder keeps up and the storage path applies no back-pressure.

The write pattern is itself favourable. The application performs one buffered write per encoded frame and flushes to the kernel, but never calls `fsync`. Data lands in the page cache and is written back in large, sequential, well-aligned extents with no barrier in the hot path; deletion happens in whole 9.45 GB files. For flash this is close to ideal. It is a log, and a log is what the medium wants.

2 Erase geometry

NAND flash rests on an asymmetry with no analogue in a hard disk. Programming moves charge in one direction only; restoring a cell to the erased state needs a high-voltage operation that array construction permits only on a whole block at once. Reads and programs address a *page*; erases address a *block* of many hundreds of pages (Fig. 1).

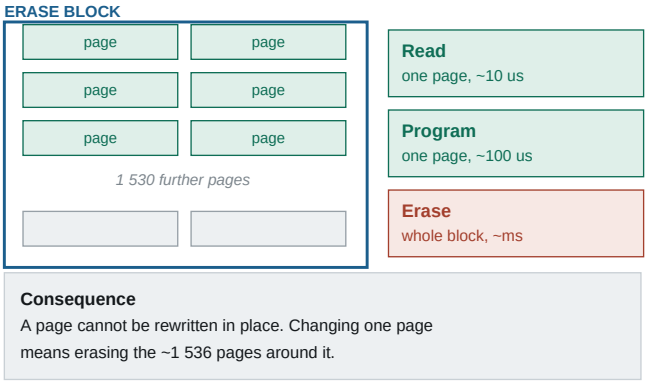

Figure 1 Granularity mismatch. Page and block sizes are representative of a 3D TLC generation; Micron's published TLC geometry gives a 16 384-byte page and 1 536 pages per block [5]. Exact values differ by part, but the ratio of three orders of magnitude between program and erase units is universal.

Each erase also wears the oxide. A block tolerates a finite number of program/erase cycles before retention degrades past what the controller's error correction can recover. Hu *et al.* quote roughly 10^5 cycles for SLC and 10^4 for the MLC of their era [1]; contemporary 3D TLC is generally specified between 1 500 and 3 000 cycles, and reaches that only with far stronger correction, on the order of 120 bits per KiB against the 40 that sufficed for planar NAND [6]. Treat any single figure as indicative.

3 Relocate-on-write and collection

Because a page cannot be overwritten in place, the drive does not try. A flash translation layer (FTL) in the controller maps the logical addresses the host uses onto the physical pages that currently hold them. A write to an already-written address is directed to a fresh pre-erased page, the map is updated, and the previous page is marked *stale*. Nothing is erased at this point.

Every page is therefore *free* (erased, available), *valid* (programmed and mapped), or *stale* (programmed but obsolete). Stale pages are not reusable, because reuse needs erasure and erasure takes the whole block. Reclaiming them is garbage collection: choose a victim, copy its surviving valid pages elsewhere, erase it (Fig. 2). Those copies are writes the host never asked for.

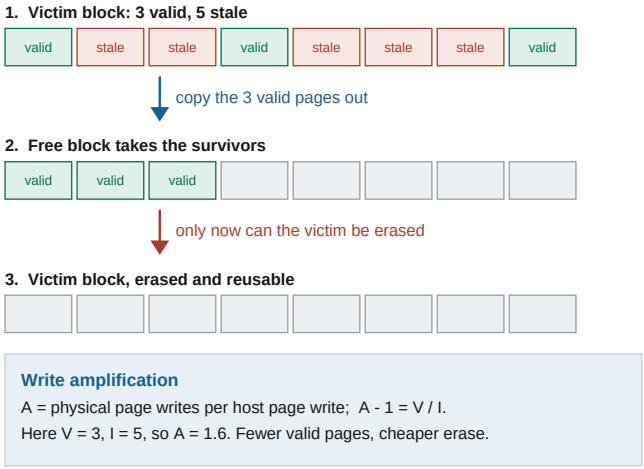

Figure 2 **The cost of reclaiming space.** The more valid pages a victim still holds, the more copying each erase requires. Keeping blocks mostly stale at collection time is the entire objective of over-provisioning and of discard. Definitions after Hu *et al.* [1].

The IBM Zurich group formalised this [1]. Write amplification A is the mean number of physical page writes per user page write, always greater than one; the amplification factor is $A - 1 = V/I$, for V valid and I invalid pages in the reclaimed block. For a raw capacity of t blocks of which the user may address u, the over-provisioning factor is $O_f = t/u$ and the spare factor is $S_f = (t - u)/t$.

Scope of that model

That analysis, and every closed form derived from it, assumes **uniformly distributed random short writes**, 4 KiB in the original. It is the adversarial case, chosen to bound the problem. The workload of Section 1 is its opposite: strictly sequential append in 629 kB records, deletion of entire multi-gigabyte files. Under that pattern a block's pages are invalidated together, V tends to zero at collection time and A tends to 1. Applying random-write amplification figures to this rig would be a category error. Discard still matters here, but for the different reason given in Section 5.

4 What `rm` does not do

There are two independent bookkeeping layers, and the word *metadata* is used for both, which is where the confusion starts.

The **filesystem** keeps an allocation bitmap and a set of inodes. These live *on* the drive, as ordinary data in ordinary logical blocks. `rm` unlinks the inode and clears bits in the bitmap: a few small writes to filesystem metadata.

The **FTL** keeps the logical-to-physical map and the valid/stale state of every page. It lives *inside* the controller, which has no interpreter for ext4. It never parses the allocation bitmap and has no idea the bitmap exists. From where it sits, `rm` is a handful of small writes somewhere in the address space, and every other address the file occupied remains exactly as valid as it was a moment earlier (Fig. 3).

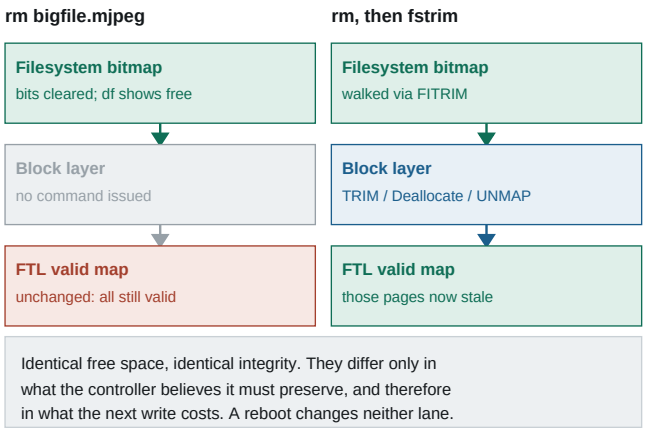

Figure 3 **Two layers, one word.** Only a discard command edits FTL metadata. The physical data remains in NAND after either path until collection happens to erase the block holding it.

This is why rebooting, or a full power cycle, changes nothing. The FTL map is itself non-volatile, journalled in NAND by the controller and reconstructed on power-up. It has to be, or no file would survive a power cut. There is no handshake at boot in which the operating system declares its free space; the only thing that ever makes that declaration is an explicit discard command.

5 Why discard still matters here

Section 3 established that sequential append with whole-file deletion drives amplification toward unity, so the usual argument for discard is weak in this case. The argument that does apply concerns the **free block pool**.

The controller writes only into pre-erased blocks, maintaining a pool of them and refilling it by collecting blocks it judges mostly stale. Without discard, deleted data is indistinguishable from live data, so across repeated fill-and-clear cycles the controller's count of valid data ratchets upward toward the full exposed capacity while `df` keeps reporting an empty filesystem. The pool is then refilled only from intrinsic spare area.

That spare is small. A drive sold as 1 TB exposes 10^{12} bytes while carrying 1 024 GiB of NAND, a spare factor S_f near 9% before any firmware reservation. Nine per cent is adequate when the host has told the truth about what is free; it is thin when the host has silently retained a claim on everything it ever wrote. The consequence is not lower mean throughput but **tail latency**: collection migrates out of idle time into the write path, appearing as occasional multi-second stalls on a write that normally completes in microseconds. This is the quantity worth protecting, and the one that benchmarks reporting mean MB/s hide.

6 Discard in practice

Three transports, three names for one request. On SATA it is the Trim function of **DATA SET MANAGEMENT**, selected by bit 0 of the feature register, with support advertised in IDENTIFY DEVICE word 169 bit 0 [2]. On NVMe it is **Dataset Management** with the Deallocate attribute, and the specification is explicit that the command is advisory: a compliant controller may take no action [3]. On SCSI, and so on USB bridges speaking UAS, it is **UNMAP**, signalled by the LBPU bit of the Logical Block Provisioning VPD page [4].

What a read of a discarded address returns is a separate question. DRAT (word 69 bit 14) promises only that repeated reads return the *same* value, not zero; RZAT (word 69 bit 5) promises zeros; without either, a read may return the old data until collection erases it [2]. For this application the distinction is academic, but it is why discard is not a sanitisation mechanism.

Batched or continuous

The `discard` mount option issues a discard inline with every deletion; `fstrim` walks the free-space map through the FITRIM ioctl and issues them in batches. Current distribution guidance favours the batched form and treats the mount option as the exception [7], chiefly because inline discard puts a variable-latency device command in the deletion path. One correction worth recording, since it is a common and plausible-sounding error: **ext4 has no `discard=async` option.** Asynchronous discard is a Btrfs feature, added there in Linux 5.6 and default for SSDs from 6.2. For ext4 the choice is `discard` or nothing, and nothing plus `fstrim` is the better one.

Most distributions ship a weekly `fstrim.timer` with `Persistent=true`, so a run missed while the machine was off is caught up after the next boot. This is almost certainly why deleting data and rebooting sometimes appears to restore performance: the timer fired, not the reboot.

The USB-attached case

This is where a Raspberry Pi deployment is most likely to be silently broken. Many USB-to-SATA and USB-to-NVMe bridges default to a SCSI provisioning mode that does not translate UNMAP, so `fstrim` errors or trims nothing and the weekly timer runs to no effect for the life of the drive. The remedy is to write `unmap` into the device's `provisioning_mode` and persist it with a udev rule keyed on the bridge's vendor and product IDs [8]. Some bridges, notably certain JMicron parts, are quirked in the kernel and cannot be made to work at

all.

```
lsblk --discard # 0 granularity = no support
sg_vpd -p lbpv /dev/sdX # LBPU=1 = accepts UNMAP
echo unmap | sudo tee \
  /sys/class/scsi_disk/*/provisioning_mode
sudo fstrim -v /path/to/mount
```

The byte count `fstrim -v` reports is the diagnostic. A figure close to the volume recently deleted means nothing had trimmed it; a figure near zero means something already had.

7 Budgets for the acquisition rig

Rated endurance is quoted as terabytes written under JEDEC JESD218, using the client trace of JESD219: a bursty, localised, eight-hours-a-day pattern with substantial idle time [9]. That model is nothing like continuous sequential acquisition and is deliberately pessimistic about amplification, so the sticker understates what this workload achieves. Taking the physical budget directly instead (Table 2), a 1 TB drive of 3D TLC rated at 3 000 cycles absorbs on the order of 3 PB of NAND writes: upwards of three thousand complete runs at unit amplification, still beyond a thousand at a pessimistic $A = 3$. At one run per week the drive is obsolete long before it is worn.

Budget	Per run	Weekly
Host writes	`0.765 TB`	`39.8 TB/yr`
NAND at A = 1.1	`0.84 TB`	`43.7 TB/yr`
NAND at A = 3	`2.30 TB`	`119 TB/yr`
P/E budget, 1 TB	`3 PB`	`decades`

TABLE 2 Endurance budget. The physical ceiling is three orders of magnitude above the demand.

Throughput is not the constraint either. A 765 GB run exceeds any plausible pseudo-SLC cache by an order of magnitude, reported dynamic caches on 1 TB consumer drives ranging from tens of gigabytes to a few hundred [10], so after the first hour the rig writes directly to TLC for the remainder of the run. Steady state is the normal state here, and burst figures on the box are irrelevant. Even so the requirement is 15.74 MB/s, against post-cache sequential rates in the hundreds of MB/s and a worst reported QLC behaviour near 100 MB/s [10]: better than sixfold headroom, consistent with the zero dropped frames of Section 1.

Where the margin actually goes

Fill level, not bitrate. A 765 GB run on a 1 TB drive ends near 77% full. Dynamic SLC allocation contracts sharply above about 80% fill [10] and free-block pressure rises with it, so the last two to three hours of an 81-split run are its most exposed part, which is also when the most accumulated experimental time stands to be lost. A 2 TB drive puts the whole run below 40% and removes this entirely, at less cost than any tuning.

How a storage stall would present

This discriminates between failure modes elsewhere in the system, so it is worth stating precisely. The encoder hands completed frames to an unbounded queue drained by one output thread. If a write stalls, that thread blocks while the encode workers continue, and the queue grows at the full 15.74 MB/s: a one-minute stall costs about 944 MB of resident memory, a five-minute stall exhausts an 8 GB Pi. A storage-induced failure therefore kills the process by memory exhaustion. It does not present as a live process sitting quietly, so any hang in which acquisition stops while the rest of the program keeps running is upstream of storage.

8 Recommended protocol

1. **Confirm discard works before trusting it.** Run `lsblk --discard` once per drive and enclosure. Zero granularity means every trim since deployment was a no-op.
2. **Trim after clearing, not on a timer.** The weekly timer is arbitrarily placed relative to a 13.5-hour run. Trim explicitly before starting, and log the byte count.
3. **Prefer `blkdiscard` when reformatting anyway.** Discarding the partition range and recreating the filesystem is faster and more complete than deleting files. Destructive, so only when starting clean.
4. **Keep ext4 mounted without `discard`.** Batched trimming is the right default.
5. **Size the drive so the run ends below 60% fill.** For 765 GB per run that means 2 TB. Highest-value change in this list, and it needs no software.
6. **Check capacity against the run, not free space.** Compute no_chunks x chunk_size_sec x 15.74 MB/s with margin at script start, and refuse to begin rather than failing eleven hours in.
7. **Leave synchronisation off during acquisition.** Concurrent reading and deletion converts a pure sequential append into a mixed pattern, competes for a CPU with no spare capacity, and introduces the output-side stall that costs memory rather than time.
8. **Record the drive's own wear counters.** `smartctl -A` reports percentage used and host writes. Logging them once per run turns speculation into a measured series.

9 Scope and limitations

This analysis does not explain the intermittent acquisition failures observed on `mjpeg_continuous_autosplit`. The evidence is against a storage cause on two independent grounds: the drive is cleared before each run and has sufficient capacity, and a storage stall would terminate the process by memory exhaustion rather than leave it running. That failure is upstream, in the camera pipeline, and is treated separately.

Several figures here are indicative rather than specific to the hardware in use. Page and block geometry varies by part; Figure 1 uses a published Micron TLC geometry [5] rather than measurements of the deployed drives. Endurance is quoted by vendors across 1 500 to 4 000 cycles for 3D TLC [6], and the 3 000 of Section 7 sits mid-range. Cache sizes and post-cache rates come from whole-drive fill tests on other drives [10]. None of this changes the conclusions, each of which carries at least an order of magnitude of margin, but a claim about a specific drive should be measured on that drive.

References

[1] Hu, X.-Y., Eleftheriou, E., Haas, R., Iliadis, I., Pletka, R. *Write Amplification Analysis in Flash-Based Solid State Drives*. SYSTOR '09, Haifa, 2009. IBM Zurich Research Laboratory.

[2] T13/INCITS. *Trim Clarifications* (e09158r1). TRIM bit at IDENTIFY DEVICE word 169 bit 0; DRAT word 69 bit 14; RZAT word 69 bit 5.

[3] NVM Express. *NVM Command Set Specification, Revision 1.0*, 2021. Dataset Management / Deallocate; DLFEAT; the advisory clause.

[4] T10. SBC-3, logical block provisioning: UNMAP and the LBPU bit of the Logical Block Provisioning VPD page (B2h). See also sg_unmap(8).

[5] Micron Technology. *Why Micron 3D NAND?* TLC geometry of 16 384 + 2 208 bytes per page, 1 536 pages per block. A 32-layer generation; representative, not current.

[6] Comparative 3D NAND endurance, including the error-correction cost of matching planar P/E counts. *The Challenges of Industrial 3D NAND*, Cyber Defense Magazine. Secondary source; vendor figures vary.

[7] Arch Wiki, *Solid state drive*; Rocky Linux, *Configuring TRIM*. Both recommend periodic fstrim over the discard mount option for general workloads.

[8] Geerling, J. *Enabling TRIM on an external SSD on a Raspberry Pi*, 2020. The provisioning_mode procedure and udev rule.

[9] JEDEC JESD218 (endurance test method) and JESD219 (workloads), committee JC-64.8. Summarised in Cox, A., *JEDEC SSD Specifications Explained*.

[10] Whole-drive sequential fill tests establishing SLC cache size and post-cache rates, e.g. AnandTech reviews of the Kioxia BG4 1 TB and ADATA SU750 1 TB.